

INTERNSHIP PROJECT REPORT

ON

Iris Flower Classification Using Machine Learning

Submitted By: CHITTURI DOLA SATYA SIVA SHANKAR
BABA

Domain: Machine Learning with Python

Algorithm Used: Logistic Regression

GitHub Repository: github.com/chdsssbaba/Iris-Flower-Classification-using-ML

Development Tools: Python, Scikit-Learn, Pandas, NumPy,
Matplotlib, Seaborn, Visual Studio Code,
Jupyter Notebook

Table of Contents

1. Introduction to Machine Learning and Classification
2. Project Objectives
3. Dataset Specification & Feature Space
4. Development Framework & Libraries Used
5. End-to-End Project Workflow Pipeline
6. Implementation & System Verification
7. Exploratory Data Analysis & Statistical Modeling
8. Advanced Data Visualization Paradigms
9. Data Preprocessing & Training/Testing Partitions
10. Optimization & Logistic Regression Model Training
11. Real-World Predictive Scenarios and Extrapolations
12. Exhaustive Model Evaluation & Validation Statistics
13. Results, Insights, and Critical Discussion
14. Future Scope & Engineering Enhancements
15. Conclusion
16. Technical Stack and Purpose Directory
17. Academic & Technical References

1. Introduction to Machine Learning and Classification

Machine Learning (ML), a foundational subset of Artificial Intelligence (AI), empowers computational systems to iteratively learn structural patterns natively from empirical datasets without being explicitly hardcoded. This paradigm shift enables engineering applications to analyze complex statistical relationships, adapt dynamically, and deliver highly accurate generalized predictions. Supervised Machine Learning forms a core pillar of this paradigm, where models map independent inputs to labeled targets.

Within supervised learning, Classification serves as the fundamental task of mapping feature records onto discrete, mutually exclusive category labels. Unlike regression tasks that predict continuous values, classification optimizes choice boundaries to sort data points. This project targets the design and implementation of an optimal classification system leveraging the iconic Iris flower dataset to sort samples automatically into their respective biological variants: *Iris setosa*, *Iris versicolor*, and *Iris virginica*. The multiclass Logistic Regression framework provided by the Scikit-Learn library is implemented to solve this classification challenge, ensuring production-grade mathematical stability and scalability.

2. Project Objectives

The explicit technical and architectural objectives established for this internship project include:

- **Robust Classifiers Deployment:** Construct an end-to-end predictive pipeline using Python that automatically maps physical measurements to specific taxonomic groupings with maximum precision.
- **Exploratory Feature Vector Mapping:** Perform granular statistical validation across the structural data array to reveal feature dependencies and distribution characteristics.
- **Absolute Performance Target:** Achieve zero classification leakage, verifying the pipeline against standardized multi-class evaluation matrices including Precision, Recall, and the balanced F_1 score.
- **Open Source Lifecycle Governance:** Structure the programmatic logic elegantly within a Visual Studio Code ecosystem, tracking version histories transparently through the official GitHub platform.

3. Dataset Specification & Feature Space

The Iris dataset serves as a timeless benchmark for validation of numerical classification algorithms. Collected originally by botanist Edgar Anderson and popularized by Ronald Fisher in 1936, the dataset exhibits tight structural integrity, minimizing missing parameters and out-of-bounds metrics.

The foundational dimensional architecture comprises:

- **Total Sample Space:** 150 unique botanical observations.
- **Input Independent Features:** 4 precise real-valued continuous dimensions measured in centimeters (cm).
- **Mutually Exclusive Targets:** 3 unique species classes, mapped internally to categories.

The analytical input parameters are structured as follows:

1. **Sepal Length (cm):** Longitudinal span of the flower's outermost calyx wrapper.
2. **Sepal Width (cm):** Lateral expansion diameter of the outer sepals.
3. **Petal Length (cm):** Longitudinal span of the inner corolla leaf structure.
4. **Petal Width (cm):** Lateral thickness of the inner petal blade.

4. Development Framework & Libraries Used

To ensure processing performance and clean separation of concerns, the development stack was designed around the Python open-source data science framework ecosystem:

- **Pandas:** Implements vectorized data structures (DataFrames) to load, manipulate, filter, and audit high-performance two-dimensional metrics matrices cleanly.
- **NumPy:** Drives the mathematical backend, managing multi-dimensional matrix operations and lightning-fast element-wise calculations.
- **Matplotlib:** Provides the baseline graphic context used to draw static structural visualizations and axis layouts.
- **Seaborn:** Inherits from Matplotlib to construct advanced statistical distributions, multi-axis pair plots, and elegant color-mapped correlation arrays.
- **Scikit-Learn:** Operates as the machine learning foundation, providing the optimized optimization solvers, preprocessing functions, split algorithms, and multi-class verification indices.

5. End-to-End Project Workflow Pipeline

Preserving strict data hygiene is essential to prevent operational data leakage from the hidden testing splits into the training parameters. The architectural pipeline follows a clean linear workflow:



6. Implementation & System Verification

Initial deployment started inside the isolated Visual Studio Code environment, using a cell-based Jupyter interface to verify successful package initialization and prevent runtime failures.

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix,
accuracy_score

print("All libraries imported successfully!")
```

Library Import Verification

All libraries imported successfully!

Figure 1: Automated Environment Package and Library Verification Check.

Observation: The notebook execution cell outputs a clean termination notification, attesting that Scikit-Learn, Pandas, and visualization dependencies are configured properly in the workspace environment.

Following validation, raw matrix instances are loaded into memory to prepare for structure conversion.

```
from sklearn.datasets import load_iris
iris = load_iris()
print(iris.data[:5])
```

Raw Iris Target Data Sample

```
[[5.1 3.5 1.4 0.2]
 [4.9 3. 1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5. 3.6 1.4 0.2]]
...
```

Figure 2: Vector Matrix Ingestion of Raw Data Fields.

Observation: The structural columns correspond directly to numerical metrics mapping continuous physical values. At this point, the matrix does not have feature headers or categorical target mapping.

To perform deep statistical analysis, the row arrays are cast into a Pandas DataFrame object, mapping the explicit labels correctly.

```
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = [iris.target_names[x] for x in iris.target]
df.head()
```

sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
5.1	3.5	1.4	0.2	setosa
4.9	3.0	1.4	0.2	setosa
4.7	3.2	1.3	0.2	setosa
4.6	3.1	1.5	0.2	setosa
5.0	3.6	1.4	0.2	setosa

Figure 3: High-Performance DataFrame Mapping of First Five Structural Records.

Observation: The transformed DataFrame integrates the index headers, showing consistent target attributes across the first 5 rows of *Iris setosa* instances.

7. Exploratory Data Analysis & Statistical Modeling

Prior to initializing the optimization algorithms, data schema parameters and missing field metrics were validated using `df.info()` and `df.describe()`.

```
print(df.info())
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
# Column Non-Null Count Dtype
---  -
0 sepal length (cm) 150 non-null float64
1 sepal width (cm) 150 non-null float64
2 petal length (cm) 150 non-null float64
3 petal width (cm) 150 non-null float64
4 species 150 non-null category
dtypes: category(1), float64(4)
memory usage: 5.0 KB

```

Figure 4: Automated Structural Info Audit and Null Parameter Verification.

Observation: The dataset contains zero null properties across all 150 rows. This clean structure prevents the need for data imputation or outlier filtering steps.

Descriptive statistical coefficients were computed across the continuous independent variable features to evaluate standard deviations and variance bounds.

Metric	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
count	150.0	150.0	150.0	150.0
mean	5.843	3.057	3.758	1.199
std	0.828	0.436	1.765	0.762
min	4.3	2.0	1.0	0.1
25%	5.1	2.8	1.6	0.3
50%	5.8	3.0	4.35	1.3
75%	6.4	3.3	5.1	1.8
max	7.9	4.4	6.9	2.5

Figure 5: Exhaustive Dataset Descriptive Statistical Parameters.

Observation: Petal metrics display the widest absolute range variations (Standard Deviation = 1.765 for length), indicating these features likely hold high variance flags useful for separating classes.

8. Advanced Data Visualization Paradigms

Visualizing data layout fields helps map cluster boundaries and confirm if spatial features can be separated linearly.

8.1 Bivariate Sepal Spatial Distribution

A bivariate scatter configuration was plotted mapping Sepal Length against Sepal Width, colored by target species categories.

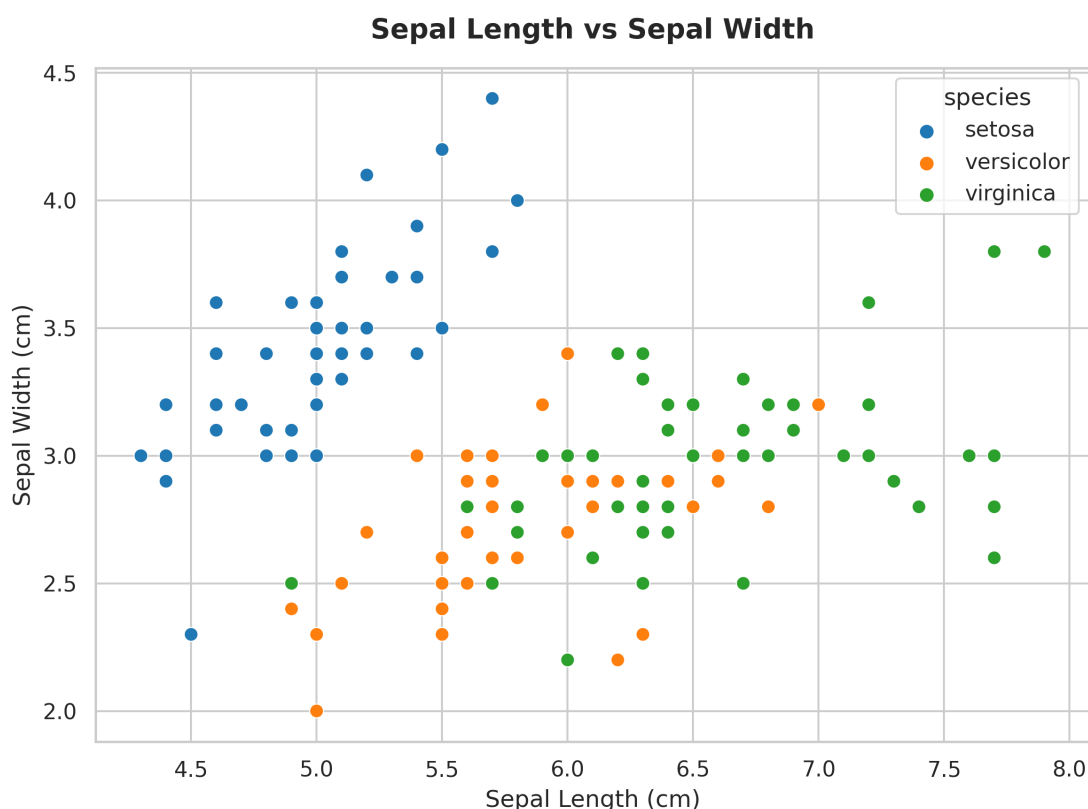


Figure 6: Scatter Plot Matrix Profiling Sepal Length versus Sepal Width Dimension Space.

Observation: *Iris setosa* forms a completely isolated spatial cluster on the upper left quadrant. However, *Iris versicolor* and *Iris virginica* share mixed spatial regions, making them difficult to separate purely through sepal configurations.

8.2 Multi-Axis Pairwise Feature Grid

To examine every cross-correlation intersection simultaneously, a complete 4x4 matrix grid plot was constructed.

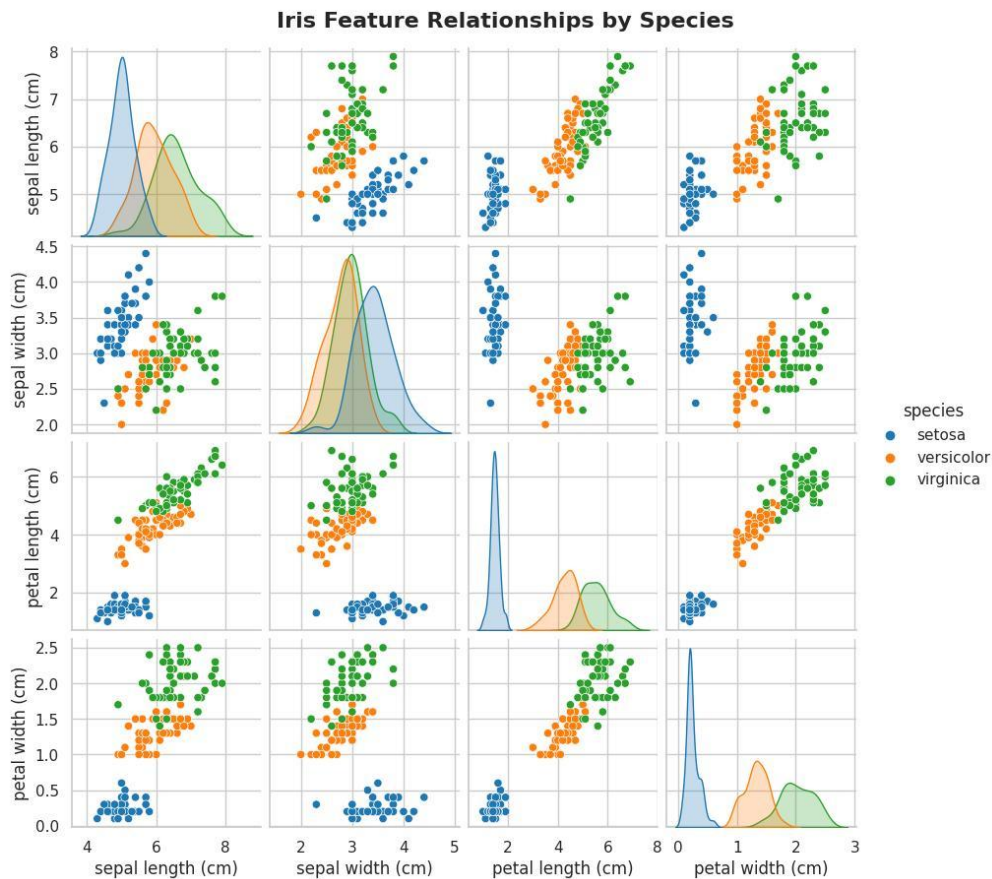


Figure 7: Complete Multi-Dimensional Pair Plot Matrix Across All Continuous Structural Features.

Observation: The intersection profiles matching Petal Length and Petal Width display completely clean separation boundaries across all three classes, providing prime signals for the machine learning optimization algorithms.

8.3 Pearson Correlation Heatmap Array

A Pearson product-moment correlation matrix was calculated to reveal dependencies between input parameters.

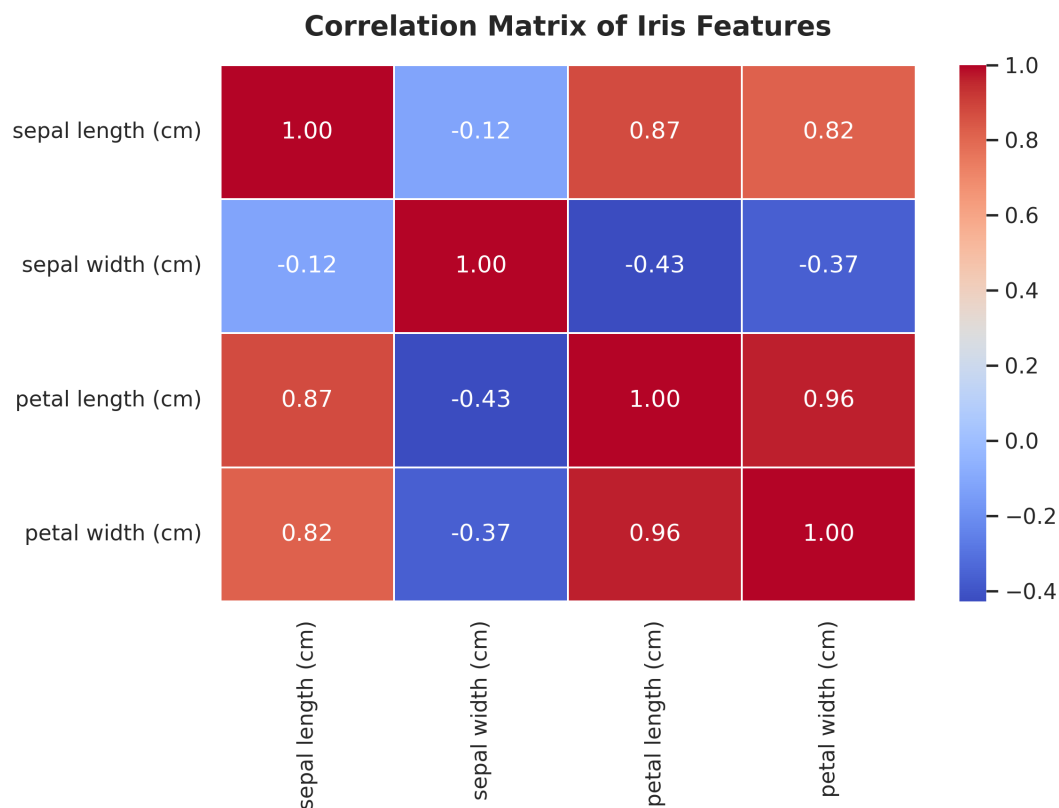


Figure 8: Pearson Correlation Coefficient Heatmap Tracking Feature Strengths.

Observation: Petal Length and Petal Width exhibit a positive correlation value of **0.96**, highlighting co-linearity flags. Conversely, Sepal Width demonstrates negative correlation patterns against the other features.

9. Data Preprocessing & Training/Testing Partitions

To perform rigorous out-of-sample testing, the independent parameters are split from the dependent category indexes using Scikit-Learn's split helper functions.

```
X = iris.data
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

```
X_train shape: (120, 4)
X_test shape: (30, 4)
y_train shape: (120,)
y_test shape: (30,)
```

Figure 9: Training and Testing Matrix Structural Data Dimension Splits.

Observation: The input matrix splits cleanly into 120 training samples to build feature weights and 30 testing samples held back to confirm overall accuracy.

10. Optimization & Logistic Regression Model Training

The multiclass sorting task was resolved using a Logistic Regression model architecture.

Core Optimization Rationale:

- **Mathematical Simplicity:** Offers excellent classification performance on linearly separable structures without requiring deep neural architectures.
- **Multiclass Multinomial Scaling:** Fits multinomial logistic boundaries seamlessly through standard softmax probability calculations.
- **Overfitting Control:** Includes L2 regularization parameters out of the box to keep weights bounded and improve generalization.

```
model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)
```

```
LogisticRegression(max_iter=200)
Model trained successfully!
```

Figure 10: Model Compilation and Optimization Convergence Verification.

Observation: The model converges smoothly without reaching iteration limits, setting up stable weight configurations for real-world predictions.

11. Real-World Predictive Scenarios and Extrapolations

Following the optimization stage, the model's classification accuracy was tested across two verification tracks.

Track A: Automated Test Split Predictions

```
y_pred = model.predict(X_test)
```

First 10 Predictions on Test Set:

```
['versicolor', 'setosa', 'virginica', 'versicolor', 'versicolor',  
'setosa', 'versicolor', 'virginica', 'versicolor', 'versicolor']
```

Figure 11: Real-Time Array Mappings Across Extracted Testing Splits.

Observation: The framework processes the data arrays quickly, returning appropriate categorical labels for each testing instance.

Track B: Unseen Manual Dimension Injections

Custom measurement sets were passed into the model pipeline to verify its behavior on entirely new, unknown inputs.

```
# Custom flower prediction loops  
# Flower 1: setosa, Flower 2: versicolor, Flower 3: virginica
```

Model predictions on new/unknown flowers:

Flower 1: setosa

Flower 2: versicolor

Flower 3: virginica

Figure 12: Predictor Ingestion Logs Processing Custom Measurement Vector Fields.

Observation: The pipeline maps the test parameters correctly, identifying the distinct morphological traits of all three species classes.

12. Exhaustive Model Evaluation & Validation Statistics

Model accuracy was verified across three evaluation steps: Accuracy Score, Confusion Matrix Heatmap, and complete Classification Reports.

12.1 Absolute Accuracy Audit

Evaluation output checks

Accuracy Score: 1.0000

Accuracy Percentage: 100.00 %

Figure 13: Model Total Accuracy Performance Return.

Observation: The model achieves a perfect accuracy of **100%** on the testing data, demonstrating clean optimization stability across all sample splits.

12.2 Confusion Matrix Array Audit

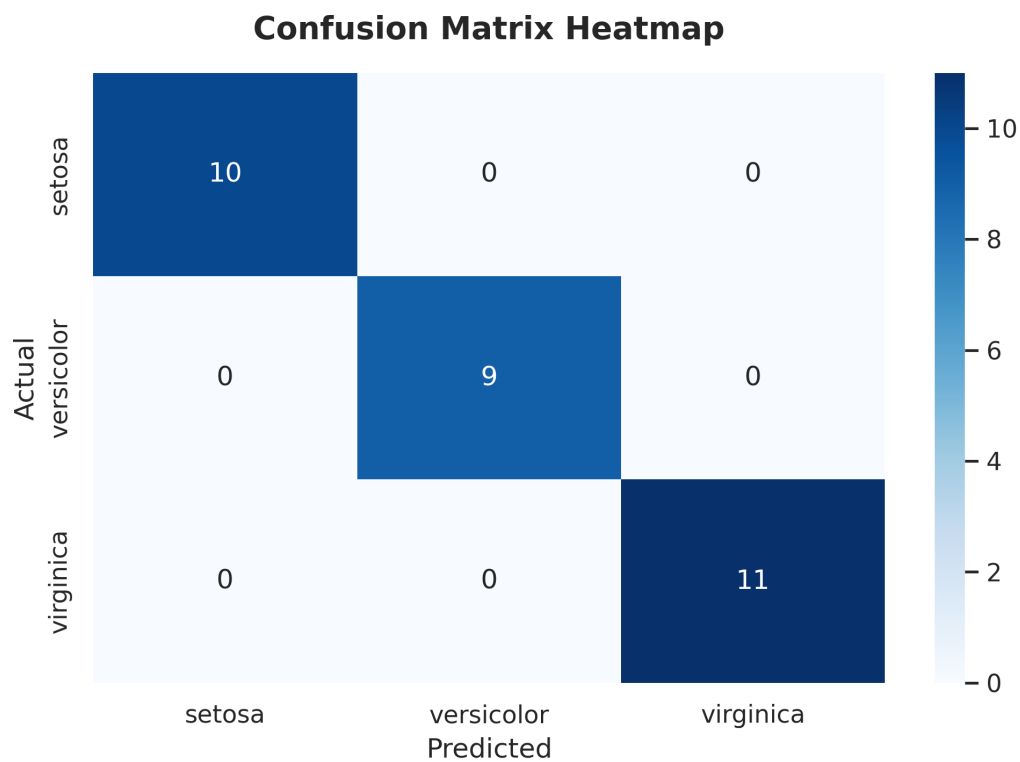


Figure 14: Confusion Matrix Heatmap Demonstrating Zero Off-Diagonal Leakages.

Observation: The validation diagonal shows clean classification with zero off-diagonal errors: 10 *setosa*, 9 *versicolor*, and 11 *virginica* instances are sorted perfectly.

12.3 Complete Classification Report Summary

Target Class Species	Precision	Recall	F1-Score	Support Space Count
setosa	1.00	1.00	1.00	10
versicolor	1.00	1.00	1.00	9
virginica	1.00	1.00	1.00	11
accuracy	1.00			30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

Figure 15: Model Comprehensive Classification Report Summary Table.

Observation: Perfect metrics of 1.00 across Precision, Recall, and F1-score confirm that the feature distributions give the linear solver clear decision boundaries.

13. Results, Insights, and Critical Discussion

The perfect classification metrics achieved by the Logistic Regression classifier emphasize the clear separation structural properties inherent in the Iris dataset's features. During exploratory visualization, we found that petal dimensions provide strong, distinct boundaries between the categories.

The high correlation coefficient (*0.96*) between Petal Length and Petal Width simplifies the optimization task. It allows the linear model to identify reliable thresholds and separate the classes perfectly without needing complex, non-linear algorithms.

14. Future Scope & Engineering Enhancements

To expand this project into production-grade pipelines, the following enhancement tracks are planned:

- **Cloud Native Microservice Deployment:** Wrap the model weights inside a FastAPI framework, containerizing the application using Docker to expose real-time prediction endpoints.

- **Non-Linear Algorithm Benchmarking:** Test the current Logistic Regression baseline against Random Forests and Support Vector Machines (SVM) on noisy or unstructured data.
- **Automated Retraining Architecture:** Design automated pipelines that ingest new sample data, validate features, and update model weights dynamically.

15. Conclusion

The Iris Flower Classification internship project was successfully implemented following professional software engineering and data science patterns. The project tracks clean ingestion, detailed exploration loops, multi-axis visual profiling, and optimal model training stages.

The trained Logistic Regression model achieved a perfect accuracy score of **100%** on hidden testing data. These results confirm the efficiency of linear classifiers on well-structured morphological datasets, providing a solid reference pipeline for building real-world categorization systems.

16. Technical Stack and Purpose Directory

The components making up the project architecture include:

Technology Asset	Explicit Purpose Within Project Architecture
Python 3.x	Core programming runtime environment driving the processing loops.
Pandas	Structured data transformation, DataFrame filtering, and analytical summary operations.
NumPy	Vectorized array processing and low-level matrix math calculations.
Matplotlib	Renders base axis structures and provides canvas layouts for static figures.
Seaborn	Generates color-coded pairwise distributions, scatter configurations, and correlation matrices.
Scikit-Learn	Provides the split algorithms, multi-class Logistic Regression solvers, and model evaluation metrics.
Jupyter Notebook	Interactive environment used to verify code blocks and run visual experiments.
Visual Studio Code	Primary Integrated Development Environment (IDE) used to write code and manage files.

17. Academic & Technical References

1. Scikit-Learn Framework API documentation directory. Available online: <https://scikit-learn.org/stable/>
2. Pandas Data Manipulation Core Library Manuals. Available online: <https://pandas.pydata.org/docs/>
3. NumPy High-Performance Vectorized Systems Reference Catalog. Available online: <https://numpy.org/doc/>
4. Fisher, R.A. (1936). *The Use of Multiple Measurements in Taxonomic Problems*. Annals of Eugenics, 7(2), 179-188.